Shared Displacement Preparation Specification

D0 Contract for Prepared Coordinates and Deterministic Block Iteration

mdstats

2026-07-22

1. Purpose and status

This document specifies the D0 shared displacement infrastructure introduced in `mdstats 0.19.81a0`. D0 does not define a new physical estimator. It centralizes the input semantics and memory-bounded displacement generation required by MSD, the self van Hove function, the non-Gaussian parameter, and the self-intermediate scattering function.

The implementation resides in `mdstats.analysis._displacement_common`. The direct time-origin-averaged MSD backend consumes this layer and remains the numerical oracle for later displacement observables. The independent FFT MSD backend is intentionally not rewritten.

The displacement identity

$$\Delta\mathbf{r}_i(t;t_0) = \mathbf{r}_i(t_0+t) - \mathbf{r}_i(t_0)$$

is standard kinematics. The preparation bundle, deterministic iteration order, block planner, metadata, and failure rules are `mdstats` design.

2. Scientific boundary

Every displacement observable must resolve the following choices exactly once:

- analyzed frame sequence and physical time grid;
- measured atom selection and its canonical order;
- laboratory or reference-cell coordinates;
- reference-cell policy and matrix;
- optional center-of-mass or center-of-geometry drift subtraction;
- exact drift-reference atom selection;
- physical analysis subspace; and
- complete `DynamicsInputSignature` provenance.

No downstream displacement observable may independently reconstruct or reinterpret these choices after receiving a prepared bundle.

3. Coordinate construction

For unwrapped fractional row coordinates $\tilde{\mathbf{s}}_{i,t}$ and cell matrix H_t, laboratory coordinates are

$$\mathbf{r}_{i,t}^{\mathrm{lab}} = \tilde{\mathbf{s}}_{i,t} H_t.$$

Reference-cell coordinates use one fixed nonsingular matrix H_{ref}:

$$\mathbf{r}_{i,t}^{\mathrm{ref}} = \tilde{\mathbf{s}}_{i,t} H_{\mathrm{ref}}.$$

Supported reference policies are `"initial"`, `"mean"`, and an explicit finite 3×3 row-vector cell matrix. The bundle stores both the resolved matrix and the policy label.

The D0 implementation preserves the existing MSD coordinate convention exactly. It does not perform periodic unwrapping; continuous fractional coordinates are a precondition of `AtomisticFrameCollection`.

4. Drift subtraction

For drift-reference set R, center-of-geometry subtraction uses

$$\mathbf{c}_R(t) = \frac{1}{|R|} \sum_{j \in R} \mathbf{r}_j(t),$$

and center-of-mass subtraction uses

$$\mathbf{c}_R(t) = \frac{\sum_{j \in R} m_j \mathbf{r}_j(t)}{\sum_{j \in R} m_j}.$$

Prepared positions are

$$\mathbf{r}'_{i,t} = \mathbf{r}_{i,t} - \mathbf{c}_R(t).$$

If drift removal is disabled, a drift selection is invalid. If it is enabled without an explicit drift selection, all atoms define the reference group. A zero or nonfinite total drift mass is rejected.

The existing `CollectiveMotionWarning` remains mandatory when the measured subset is also the complete drift-reference subset while not containing all trajectory atoms.

5. Analysis subspace

The prepared bundle stores an `AnalysisSubspace` with orthonormal row basis

$$B \in \mathbb{R}^{d \times 3}, \qquad BB^T = I_d, \qquad d \in \{1, 2, 3\}.$$

The iterator returns projected displacement samples

$$\Delta \mathbf{s}_i(t; t_0) = B[\mathbf{r}'_i(t_0 + t) - \mathbf{r}'_i(t_0)].$$

Axis order is meaningful for the returned coordinate array. For example, `axes=("y", "x")` returns components in that order. The bundle signature must contain the exact same basis and labels, not merely an equivalent projector.

6. Prepared input bundle

```
@dataclass(frozen=True, slots=True)
class DisplacementInputBundle:
    positions: NDArray[np.float64]             # (T, M, 3)
    times_ps: NDArray[np.float64]              # (T,)
    sample_spacing_ps: float
    atom_indices: NDArray[np.int64]            # (M,)
    coordinate_mode: str
    reference_cell_mode: str | None
    reference_cell: NDArray[np.float64] | None # (3, 3)
    drift_mode: str | None
    drift_atom_indices: NDArray[np.int64] | None
    subspace: AnalysisSubspace
    signature: DynamicsInputSignature
    metadata: Mapping[str, Any]
```

The bundle owns its prepared position array. All arrays are C-contiguous and read-only; metadata is recursively immutable. Explicit atom-index selections retain user order. Species selections retain canonical trajectory order.

Required identities include:

- `positions.shape == (signature.n_frames, len(atom_indices), 3)`;
- `times_ps == signature.frame_times_ps` exactly;
- `sample_spacing_ps == signature.sample_spacing_ps`;
- measured atoms, drift atoms, coordinate mode, reference cell, and projection basis agree with the signature; and
- all numeric values are finite.

The position array is prepared once so later lag iteration performs no species, cell, or drift resolution.

7. Preparation API

```python
def prepare_displacement_inputs(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    coordinate_mode: Literal["laboratory", "reference_cell"] = "laboratory",
    reference_cell: ReferenceCellInput = "initial",
    drift_mode: Literal["center_of_mass", "center_of_geometry"] | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
) -> DisplacementInputBundle:
    ...
```

The input must be a trajectory with at least two frames and a strictly increasing, uniformly sampled physical time axis. D0 rejects an ensemble even if it carries numeric frame labels.

Laboratory coordinates on an appreciably varying cell emit `VariableCellMSDWarning`, preserving the implemented MSD interpretation boundary.

8. Lag contract

`lag_steps` is a nonempty one-dimensional integer array of saved-frame lags. It must be strictly increasing, unique, and satisfy

$$0 \le k < T.$$

Floating-point values that happen to be integral are rejected. Boolean values are also rejected. Lag zero is valid and produces exact zero displacement.

For uniform sample spacing Δt, the block time is

$$t_k = k\Delta t.$$

9. Block plan

```python
@dataclass(frozen=True, slots=True)
class DisplacementBlockPlan:
    atom_block_size: int
    origin_block_size: int
    bytes_per_sample: int
    estimated_peak_work_bytes: int
    memory_target_bytes: int | None
```

`atom_block_size` and `origin_block_size` are upper bounds. Values larger than the available atom or origin counts are clipped to those counts.

The conservative peak-work estimate counts:

- two gathered Cartesian endpoint arrays: 6 `float64` values;
- one Cartesian difference: 3 values;
- one projected work array: d values; and
- one owned immutable projected output: d values.

Thus

$$b_{\text{sample}} = 8(9 + 2d) \text{ bytes}.$$

For resolved block sizes A_b and O_b,

$$b_{\text{peak}} = A_b O_b b_{\text{sample}}.$$

The estimate is conservative and intentionally excludes small index arrays and Python object overhead.

9.1 Memory-target resolution

If `memory_target_bytes` is omitted, explicit block bounds or complete available dimensions are used. If a target is supplied:

1. validate that at least one displacement sample fits;
2. retain the resolved atom block when one origin fits;
3. reduce the origin block to the largest count within the target; and
4. reduce the atom block only when one origin with the requested atom block cannot fit.

The target is a hard bound on the stated numerical work estimate. Resolution is deterministic and independent of trajectory values.

10. Block iterator

```
def iter_displacement_blocks(
    bundle: DisplacementInputBundle,
    lag_steps: ArrayLike,
    *,
    origin_stride: int = 1,
    atom_block_size: int | None = None,
    origin_block_size: int | None = None,
    memory_target_bytes: int | None = None,
) -> Iterator[DisplacementBlock]:
    ...
```

The normative block schema is

```
@dataclass(frozen=True, slots=True)
class DisplacementBlock:
    lag_index: int
    lag_step: int
    lag_time_ps: float
    origin_indices: NDArray[np.int64]       # (O,)
    atom_indices: NDArray[np.int64]         # (A,)
    displacements: NDArray[np.float64]      # (O, A, d)
    n_samples: int                          # O * A
```

Iteration order is strictly:

1. input lag order;
2. increasing origin-block order; and
3. measured-atom order within increasing atom blocks.

For lag k, valid origins are

$$\mathcal{O}_k = \{0, s, 2s, \ldots\} \cap [0, T - k - 1],$$

where s is `origin_stride`. Every valid `(lag, origin, measured atom)` sample appears exactly once. No padding samples are emitted.

The block owns read-only index and displacement arrays. `n_samples` must equal `len(origin_indices) * len(atom_indices)`.

11. Direct MSD migration

The direct time-origin-averaged MSD backend prepares one full Cartesian D0 bundle, iterates displacement blocks, accumulates raw sums, and normalizes only after all blocks for a lag are complete.

For each lag,

$$M_{\alpha\beta}(k) = \frac{1}{N_{\mathrm{orig}}(k) M} \sum_{n \in \mathcal{O}_k} \sum_{i=1}^{M} \Delta r_{i\alpha}(n,k) \Delta r_{i\beta}(n,k).$$

The component curves are the diagonal, the scalar MSD is the trace, and per-atom values average squared norms over origins only.

The default direct path uses a 256 MiB displacement-work target. Its resolved atom/origin sizes and conservative peak estimate are stored in `MSDResult` metadata. The existing public `atom_block_size` option remains the FFT control; D0 block-shape controls are internal in this release.

The FFT backend retains its separate autocorrelation/prefix-sum algebra. It must continue to agree with the D0 direct oracle within floating-point tolerance.

12. Validation and failure rules

D0 must reject:

- non-trajectory collections or fewer than two frames;
- missing, nonmonotonic, or nonuniform times;
- empty, duplicate, or out-of-range measured/drift selections;
- conflicting species and explicit-index selectors;
- invalid coordinate, drift, reference-cell, or subspace definitions;
- nonfinite prepared coordinates;
- noninteger, duplicate, decreasing, negative, or unavailable lags;
- boolean or nonpositive stride/block/memory controls;
- a memory target smaller than one estimated displacement sample; and
- inconsistent manually constructed bundles, plans, or blocks.

13. Required tests

The D0 focused suite must include:

1. explicit atom-order preservation;
2. axis-order and rotated-subspace projection values;
3. lag-major/origin-major/atom-major deterministic iteration;
4. exact coverage without missing or duplicate samples;
5. zero-lag samples;
6. laboratory versus reference-cell preparation;
7. center-of-mass and center-of-geometry drift subtraction;
8. strict lag and option validation;
9. immutable bundles and blocks;
10. deterministic memory-plan resolution and hard target compliance;
11. direct MSD agreement with the pre-D0 algebra under several atom/origin block shapes;
12. per-atom, component, tensor, and scalar consistency;
13. FFT-versus-D0 direct regression; and
14. complete D0 metadata and signature propagation from `compute_msd()`.

14. Implementation boundary

D0 is complete when:

- `_displacement_common.py` implements the bundle, plan, block, preparation, planner, and iterator contracts;
- the direct time-origin-averaged MSD consumes the iterator;
- the FFT backend remains independent;
- the focused MSD/VACF/diffusion regression set passes; and
- the architecture manual and MSD specification identify D0 as the shared foundation consumed by D1-D3.

D0 itself does not implement radial histograms, non-Gaussian moments, self-intermediate scattering, conductivity, automatic diffusion fitting, or irregular-time displacement binning. D1 adds the radial self van Hove histogram without changing the D0 infrastructure contract.